Hande Eryılmaz
21902678

Lab 5: Seven-Segment Display

The goal of this lab is to use multiple seven-segment displays at the same time. I created a design that makes use of the seven-segment display on my Basys 3 while allowing me to show multiple different digits on the Basys3 board at the same time.

Process

To complete this lab successfully, a multiplexer, decoder, and counter are designed in the circuit. I wrote code for each component and organized them into a single module.

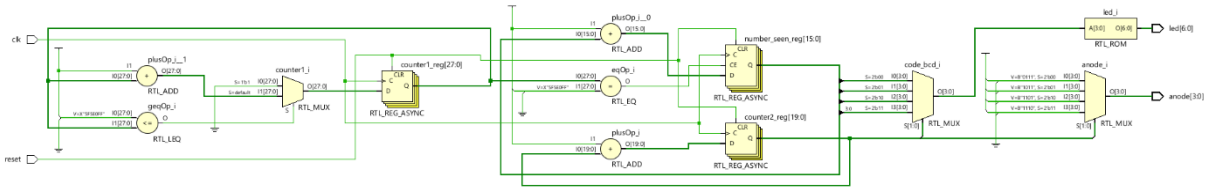

Figure 1: Schematic of the design

Basys3's internal clock frequency is 100MHz. A clock divider can be used to generate a slower clock signal from this one. A clock divider is a device that takes an input clock and converts it into an output clock. The output clock frequency is calculated by dividing the input frequency by an integer. Only lower values can be observed because the number used to divide the clock frequency is an integer. For example, if we choose 100 as the integer and use the internal clock frequency as the input frequency, we can get 1MHz as the output frequency. Alternatively, if we choose the integer 1, we will get a clock frequency of 100MHz. The interval frequency that can be obtained from clock divider is [-(input clock frequency)/ 1, (input clock frequency)/1]

Observations

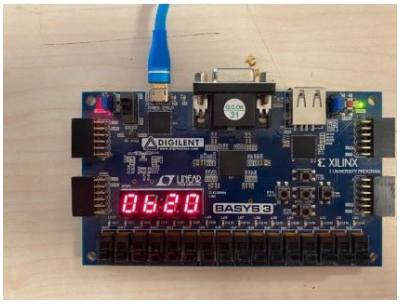

Figure 2: Before reset

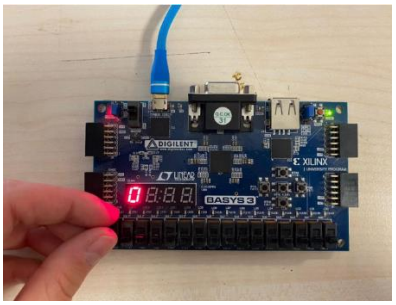

Figure 3: When reset

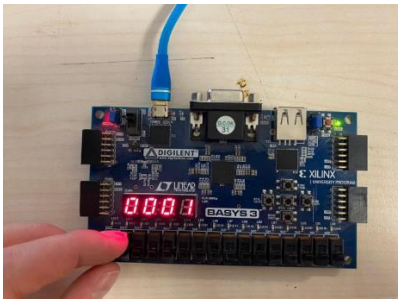

Figure 4: After reset

Hande Eryılmaz
21902678

Codes

Top Module

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.std_logic_unsigned.all;
entity seven_segment_display is
   Port ( clk : in STD_LOGIC;
          reset : in STD_LOGIC;
          anode : out STD_LOGIC_VECTOR (3 downto 0);
          led : out STD_LOGIC_VECTOR (6 downto 0));
end seven_segment_display;

architecture Behavioral of seven_segment_display is
signal counter1: STD_LOGIC_VECTOR (27 downto 0);
signal enable1: std_logic;
signal number_seen: STD_LOGIC_VECTOR (15 downto
0);
signal code_bcd: STD_LOGIC_VECTOR (3 downto 0);
signal counter2: STD_LOGIC_VECTOR (19 downto 0);
signal led_counter: std_logic_vector(1 downto 0);

begin
process(code_bcd)
begin
   case code_bcd is
   when "0000" => led <= "0000001"; -- "0"
   when "0001" => led <= "1001111"; -- "1"
   when "0010" => led <= "0010010"; -- "2"
   when "0011" => led <= "0000110"; -- "3"
   when "0100" => led <= "1001100"; -- "4"
   when "0101" => led <= "0100100"; -- "5"
   when "0110" => led <= "0100000"; -- "6"
   when "0111" => led <= "0001111"; -- "7"
   when "1000" => led <= "0000000"; -- "8"
   when "1001" => led <= "0000100"; -- "9"
   when "1010" => led <= "0000010"; -- a
   when "1011" => led <= "1100000"; -- b
   when "1100" => led <= "0110001"; -- C
   when "1101" => led <= "1000010"; -- d
   when "1110" => led <= "0110000"; -- E
   when "1111" => led <= "0111000"; -- F
   end case;
end process;

process(clk,reset)
begin
   if(reset='1') then
      counter2 <= (others => '0');
   elsif(rising_edge(clk)) then
      counter2 <= counter2 + 1;
   end if;
end process;
 led_counter <= counter2(19 downto 18);
process(led_counter)
begin
   case led_counter is
   when "00" =>
      anode <= "0111";
      code_bcd <= number_seen(15 downto 12);
   when "01" =>
      anode <= "1011";
      code_bcd <= number_seen(11 downto 8);
   when "10" =>
      anode <= "1101";
      code_bcd <= number_seen(7 downto 4);
   when "11" =>
      anode <= "1110";
      code_bcd <= number_seen(3 downto 0);
   end case;
end process;

process(clk, reset)
begin
      if(reset='1') then
         counter1 <= (others => '0');
      elsif(rising_edge(clk)) then
         if(counter1>=x"5F5E0FF") then
            counter1 <= (others => '0');
         else
            counter1 <= counter1 + "0000001";
         end if;
      end if;
end process;
enable1 <= '1' when counter1=x"5F5E0FF" else '0';
process(clk, reset)
begin
      if(reset='1') then
         number_seen <= (others => '0');
      elsif(rising_edge(clk)) then
          if(enable1='1') then
            number_seen <= number_seen + x"0001";
          end if;
      end if;
end process;
end Behavioral;
```

Testbench

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
entity testbench_top is
--  Port ( );
end testbench_top;

architecture Behavioral of testbench_top is
component seven_segment_display
   Port ( clk : in STD_LOGIC;-- 100Mhz clock on Basys 3 FPGA
board
        reset : in STD_LOGIC; -- reset
        anode : out STD_LOGIC_VECTOR (3 downto 0);-- 4 Anode
signals
        led : out STD_LOGIC_VECTOR (6 downto 0));-- Cathode
patterns of 7-segment display
end component;

signal led2: std_logic_vector(6 downto 0);
signal reset1: std_logic:='0';
signal clk: std_logic:='0';
signal anode2: std_logic_vector(3 downto 0);

begin
uut: seven_segment_display port map(
led => led2,
reset=> reset1,
clk=> clk,
anode =>anode2
 );
```

Hande Eryılmaz
21902678

```
 U1: process
 begin
 clk <='0';
 wait for 10ns;
 clk <='1';
 wait for 10ns;
 end process;
 U2:process
begin
reset1 <= '1' ;
wait for 100ns;
reset1 <='0';
wait;
end process;
end Behavioral;
```

Constraint

```
# Clock signal
set_property PACKAGE_PIN W5 [get_ports clk]
 set_property IOSTANDARD LVCMOS33 [get_ports clk]
set_property PACKAGE_PIN R2 [get_ports reset]
      set_property IOSTANDARD LVCMOS33 [get_ports reset]
#seven-segment LED display
      set_property PACKAGE_PIN W7 [get_ports {led[6]}]
        set_property IOSTANDARD LVCMOS33 [get_ports
{led[6]}]
      set_property PACKAGE_PIN W6 [get_ports {led[5]}]
        set_property IOSTANDARD LVCMOS33 [get_ports
{led[5]}]
      set_property PACKAGE_PIN U8 [get_ports {led[4]}]
        set_property IOSTANDARD LVCMOS33 [get_ports
{led[4]}]
      set_property PACKAGE_PIN V8 [get_ports {led[3]}]
        set_property IOSTANDARD LVCMOS33 [get_ports
{led[3]}]
      set_property PACKAGE_PIN U5 [get_ports {led[2]}]
        set_property IOSTANDARD LVCMOS33 [get_ports
{led[2]}]
      set_property PACKAGE_PIN V5 [get_ports {led[1]}]
        set_property IOSTANDARD LVCMOS33 [get_ports
{led[1]}]
      set_property PACKAGE_PIN U7 [get_ports {led[0]}]
        set_property IOSTANDARD LVCMOS33 [get_ports
{led[0]}]
      set_property PACKAGE_PIN U2 [get_ports {anode[0]}]
        set_property IOSTANDARD LVCMOS33 [get_ports
{anode[0]}]
      set_property PACKAGE_PIN U4 [get_ports {anode[1]}]
        set_property IOSTANDARD LVCMOS33 [get_ports
{anode[1]}]
      set_property PACKAGE_PIN V4 [get_ports {anode[2]}]
        set_property IOSTANDARD LVCMOS33 [get_ports
{anode[2]}]
      set_property PACKAGE_PIN W4 [get_ports {anode[3]}]
        set_property IOSTANDARD LVCMOS33 [get_ports
{anode[3]}]
```